CBOW training: the connections change, but the weights persist

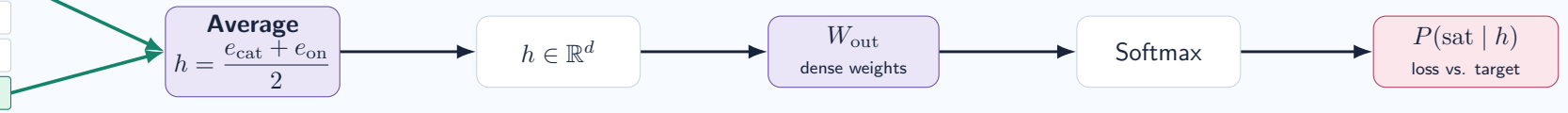
Each context window chooses rows from the SAME trainable embedding table. Backpropagation updates the rows used by that window.

1. Forward pass: sample A

context: **cat** … **on** target: **sat**

W_{in}: permanent embedding table
Word index ⟶ row of trainable weights

cat	e_{cat}	selected
dog	e_{dog}	trainable vector
sat	e_{sat}	trainable vector
on	e_{on}	selected
in	e_{in}	trainable vector
mat	e_{mat}	trainable vector



Fixed coefficients, no trainable weights

Same matrix every sample

2. Backpropagation / SGD on sample A

$$\frac{\partial L}{\partial e_{\text{cat}}} = \frac{\partial L}{\partial e_{\text{on}}} = \frac{1}{2}\frac{\partial L}{\partial h}$$

Inactive input rows (dog, sat, in, mat) receive no gradient in this example.



Parameters updated by SGD:
Selected input vectors: e_{cat} and e_{on}
Shared output matrix: W_{out}

3. Next training sample: sample B

context: **dog** … **in** target: **sat**

W_{in}: the same table, now updated
Different active rows; persistent parameters

cat	e_{cat}	trainable vector
dog	e_{dog}	selected
sat	e_{sat}	trainable vector
on	e_{on}	trainable vector
in	e_{in}	selected
mat	e_{mat}	trainable vector



Again, a fixed average

Learned from sample A and earlier samples

The MLP interpretation

Each word is a one-hot vector x_w. The embedding lookup is just $e_w = W_{\text{in}}^T x_w$.

The apparent "rewiring" is sparse row selection, not a change in the model architecture.

Only the active input embeddings change per sample; both parameter tables remain stored and trainable.

CBOW: each word carries a trainable d-dimensional embedding

Only the currently selected vectors are expanded below. Entries line up by coordinate, so the average can be read column by column.

1. Sample A: cat and on are selected

context: **cat** ⋯ **on** target: **sat**

W_{in}: persistent embedding table

cat	$\boldsymbol{e}_{\text{cat}}$	selected
dog	$\boldsymbol{e}_{\text{dog}}$	
sat	$\boldsymbol{e}_{\text{sat}}$	
on	$\boldsymbol{e}_{\text{on}}$	selected
in	$\boldsymbol{e}_{\text{in}}$	
mat	$\boldsymbol{e}_{\text{mat}}$	

Selected embeddings, and their component-wise average

d coordinates in every row

vector	1	2	⋯	d
$\boldsymbol{e}_{\text{cat}}$	0.218	−1.572	⋯	0.294
$\boldsymbol{e}_{\text{on}}$	−0.418	0.732	⋯	1.106
$\boldsymbol{h}_A$ **(avg)**	−0.100	−0.420	⋯	0.700

W_{out} + softmax

scores for all words

$\boldsymbol{h}_A \in \mathbb{R}^d$: the average preserves every coordinate (it is not a scalar).

2. Backpropagation on sample A

$\nabla_{\boldsymbol{h}_A} L = [\,0.200,\ -0.100,\ \ldots,\ 0.500\,] \in \mathbb{R}^d$

$\nabla_{\boldsymbol{e}_{\text{cat}}} L = \nabla_{\boldsymbol{e}_{\text{on}}} L = \frac{1}{2} \nabla_{\boldsymbol{h}_A} L$

SGD updates the two selected embedding rows using these gradients.

W_{out} is also trained; unused input rows do not change on this step.

3. Sample B: different rows, same network

context: **dog** ⋯ **in** target: **sat**

W_{in}: persistent embedding table

cat	$\boldsymbol{e}_{\text{cat}}$	
dog	$\boldsymbol{e}_{\text{dog}}$	selected
sat	$\boldsymbol{e}_{\text{sat}}$	
on	$\boldsymbol{e}_{\text{on}}$	
in	$\boldsymbol{e}_{\text{in}}$	selected
mat	$\boldsymbol{e}_{\text{mat}}$	

Selected embeddings, and their component-wise average

d coordinates in every row

vector	1	2	⋯	d
$\boldsymbol{e}_{\text{dog}}$	0.604	−0.280	⋯	0.016
$\boldsymbol{e}_{\text{in}}$	−0.344	0.900	⋯	−0.336
$\boldsymbol{h}_B$ **(avg)**	0.130	0.310	⋯	−0.160

W_{out} + softmax

scores for all words

$\boldsymbol{h}_B \in \mathbb{R}^d$: the average preserves every coordinate (it is not a scalar).

Key idea Each $\boldsymbol{e}_w$ is one fixed-size, trainable d-vector stored in W_{in}.

$\boldsymbol{h} = (\boldsymbol{e}_{w_1} + \cdots + \boldsymbol{e}_{w_C})/C \in \mathbb{R}^d$: average corresponding components, not the entire vector to one value.

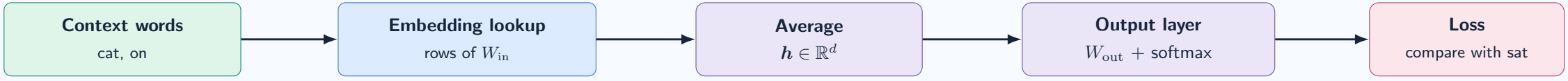
New context window = new selected rows; the stored weights and the architecture persist between training examples.

CBOW: from prediction task to word embeddings

Continuous Bag of Words learns one reusable vector per word by predicting a missing word from its context.

1. A prediction task creates the training signal

Example: context = **cat** and **on**; the correct target is **sat**.



The averaging operation has no trainable parameters; the input embeddings and output weights are trainable.

Backpropagation travels from the loss through W_{out} and $\boldsymbol{h}$ to the selected rows of W_{in}.

2. What the training actually changes

Input embedding matrix W_{in}

One learned d-vector per vocabulary word.

Only selected input rows receive a gradient in a step.

Prediction layer W_{out}

Scores candidate target words given $\boldsymbol{h}$.

Its trainable weights receive gradients as well.

3. After training: keep the product, not the predictor

KEEP W_{in}
persistent word embeddings

USUALLY DISCARD W_{out} + softmax
the training-time prediction head

The learned vectors can be reused in downstream models; normally no averaging or target prediction is needed to look up a word vector.

Main idea We train CBOW as a word predictor because prediction errors teach us useful word representations.

The lasting artifact is the embedding table; the output classifier is primarily a means of learning it.

CBOW as matrix multiplication: seven context words

The binary selection vector chooses seven distinct context words; the factor 1/7 performs the averaging.

1. Select seven words, then normalize for averaging

Example: "small cat quietly [sat] on the soft mat"; the target is sat, $C = 7$ and $V = 9$.

small	cat	quietly	sat	on	the	soft	mat	dog
1	1	1	0	1	1	1	1	0

$\boldsymbol{s} = [1, 1, 1, 0, 1, 1, 1, 1, 0]$: **binary selection vector,** $1 \times V$.

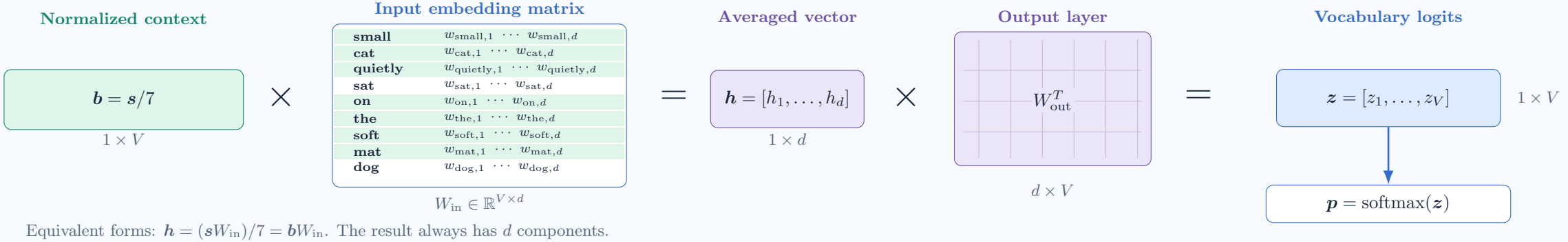
Averaging is a separate scale factor

$$\boldsymbol{b} = \frac{\boldsymbol{s}}{C} = \frac{1}{7}[1, 1, 1, 0, 1, 1, 1, 1, 0]$$

$$\boldsymbol{h} = \frac{\boldsymbol{s} W_{\text{in}}}{7} = \boldsymbol{b} W_{\text{in}}$$

Seven ones $\Rightarrow \sum_i s_i = 7$. Repeated tokens would require counts.

2. Forward pass: matrix multiplication and dimensions



Equivalent forms: $\boldsymbol{h} = (\boldsymbol{s} W_{\text{in}})/7 = \boldsymbol{b} W_{\text{in}}$. The result always has d components.

3. Why input-embedding gradients are sparse

$\boldsymbol{g} = \frac{\partial L}{\partial \boldsymbol{h}} \in \mathbb{R}^{1 \times d}$

$$\frac{\partial L}{\partial W_{\text{in}}} = \underbrace{\boldsymbol{b}^T}_{V \times 1} \underbrace{\boldsymbol{g}}_{1 \times d}$$

Outer product: the matrix gradient has shape $V \times d$.

The nine input-embedding gradient rows

Selected 7 words: $\boldsymbol{g}/7$ per row

Unselected 2 words (sat, dog): **0** per row

Only the seven selected words receive input-embedding gradients.

W_{out} also trains on this step.

One-line summary

$$\boldsymbol{p} = \text{softmax}\left(\tfrac{1}{7} \boldsymbol{s} W_{\text{in}} W_{\text{out}}^T\right)$$

Everything before softmax is linear; $W_{\text{in}} W_{\text{out}}^T$ has rank at most d.

Standard full-softmax CBOW shown. After training, typically retain W_{in} and discard the output prediction layer.

CBOW and multinomial logistic regression

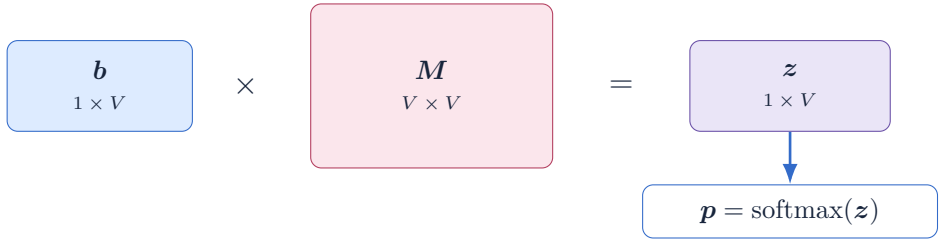
Both perform softmax classification. CBOW learns a factorized weight matrix and retains its intermediate embeddings.

1. The same classification problem, with two parameterizations

In both diagrams, $\boldsymbol{b} \in \mathbb{R}^{1\times V}$ is the normalized bag-of-words context input.

Multinomial logistic regression

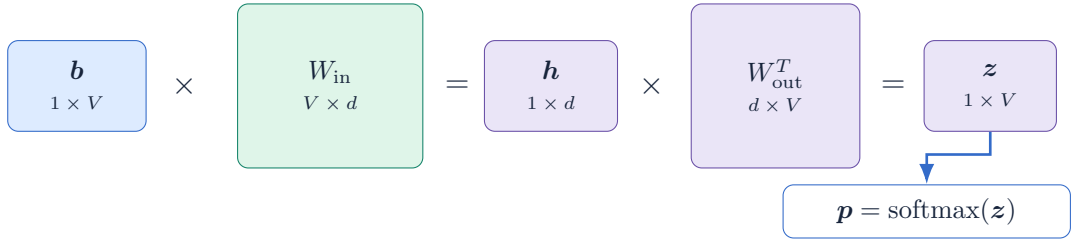
One $V \times V$ output-weight matrix



$\boldsymbol{p} = \text{softmax}(\boldsymbol{b}M + \boldsymbol{c})$ (bias $\boldsymbol{c}$ optional)

CBOW with full softmax

Two matrices joined through a d-dimensional bottleneck



$\boldsymbol{p} = \text{softmax}(\boldsymbol{b}W_{\text{in}}W_{\text{out}}^T)$ (no bias in standard CBOW)

2. The key relationship: a low-rank factorization

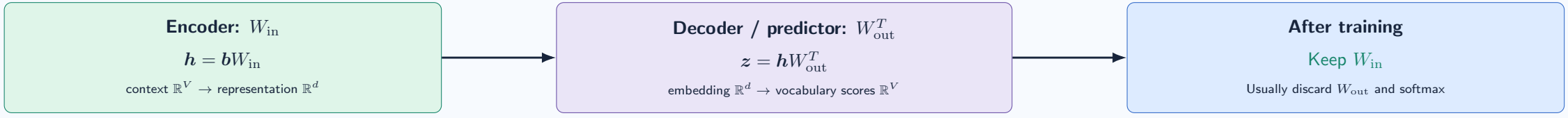
Softmax regression learns the full $V \times V$ matrix M; CBOW represents it as two smaller trainable matrices:

$$M = W_{\text{in}}W_{\text{out}}^T$$

$\text{rank}(M) \le d$; with $d \ll V$, CBOW has far fewer parameters and forces information through a d-dimensional representation.

3. Encoder / decoder interpretation

CBOW predicts the target word from its context; unlike an autoencoder, it does not attempt to reconstruct the input.



Both matrices learn through backpropagation, but the trained W_{in} supplies the word vectors we normally want to reuse.

The prediction network is a training scaffold: the learned embedding dictionary is the lasting product.